

Topics to be covered

- ✓ ~~Database administrator & Database Users~~
 - ✓ ~~Characteristics of the Database~~
 - ☐ Database Systems
 - ☐ Data Models
 - ✓ ~~Schemes & Instances~~
 - ✓ ~~DBMS Architecture & Data Independence~~
 - ✓ ~~Database Languages & Interfaces~~
 - ✓ ~~DDL (Data Definition Language)~~
 - ✓ ~~DML (Data Manipulation Language)~~
 - ✓ ~~DCL (Data Control Language)~~
-

Database Administrator and Database Users

Database Administrator

A database administrator (DBA for short) is a crucial role that manages and maintains database systems, ensuring they operate efficiently, securely, and reliably.

Key responsibilities for DBA:

1. Installation and Configuration

- Opting for the best suited Database for an organization based on the services it's providing and

what kind of data it will be storing.

- Configuring to ensure everything is set up correctly so that there are no hiccups.

2. Database Design

- It is the process of creating a detailed model of the database.
- This data model includes all the logical and physical design choices required to ensure the database can effectively and efficiently store, manage, and retrieve data for its intended purpose.

3. User Management and Capacity Planning

- Managing permissions for different users and defining what part of the database they have access to.
- Capacity planning ensures that the system is able to scale up if the organization grows in size.

4. Data Management

- It ensures that employees and other entity have a robust way to interact with the database.

5. Performance Monitoring

- Monitoring system performance during peak hours to ensure that the system is not overloaded.
- Making meaningful changes to the system to squeeze the last bit of performance.

6. Backup and Recovery

- Ensuring that there are multiple backups of the original data available.
- It ensures minimum downtime in case of a failure while ensuring that data is not lost.

7. Security

- Making sure that unauthorized users do not have access to the database.
- Introducing authentication, authorization and other methods to ensure only allowed users are able to interact with the database.

Database Users

Database users are individuals or applications that interact with a database to retrieve, store, and manage data.

Types of Database Users:

1. **End Users:** These are the people who use applications that interact with the database.
2. **Application Developers:** These are the programmers who create the applications to interact with the database.
3. **Database Administrators:** DBAs are responsible for managing and maintaining the database system.
4. **Data Analysts/Business Intelligence Users:** These users analyse the data inside the database to identify trends, patterns, and insights that can help organizations make better decisions.

Levels of Access and Permission:

- Read only access
 - Read-Write access
 - Administrative access
-

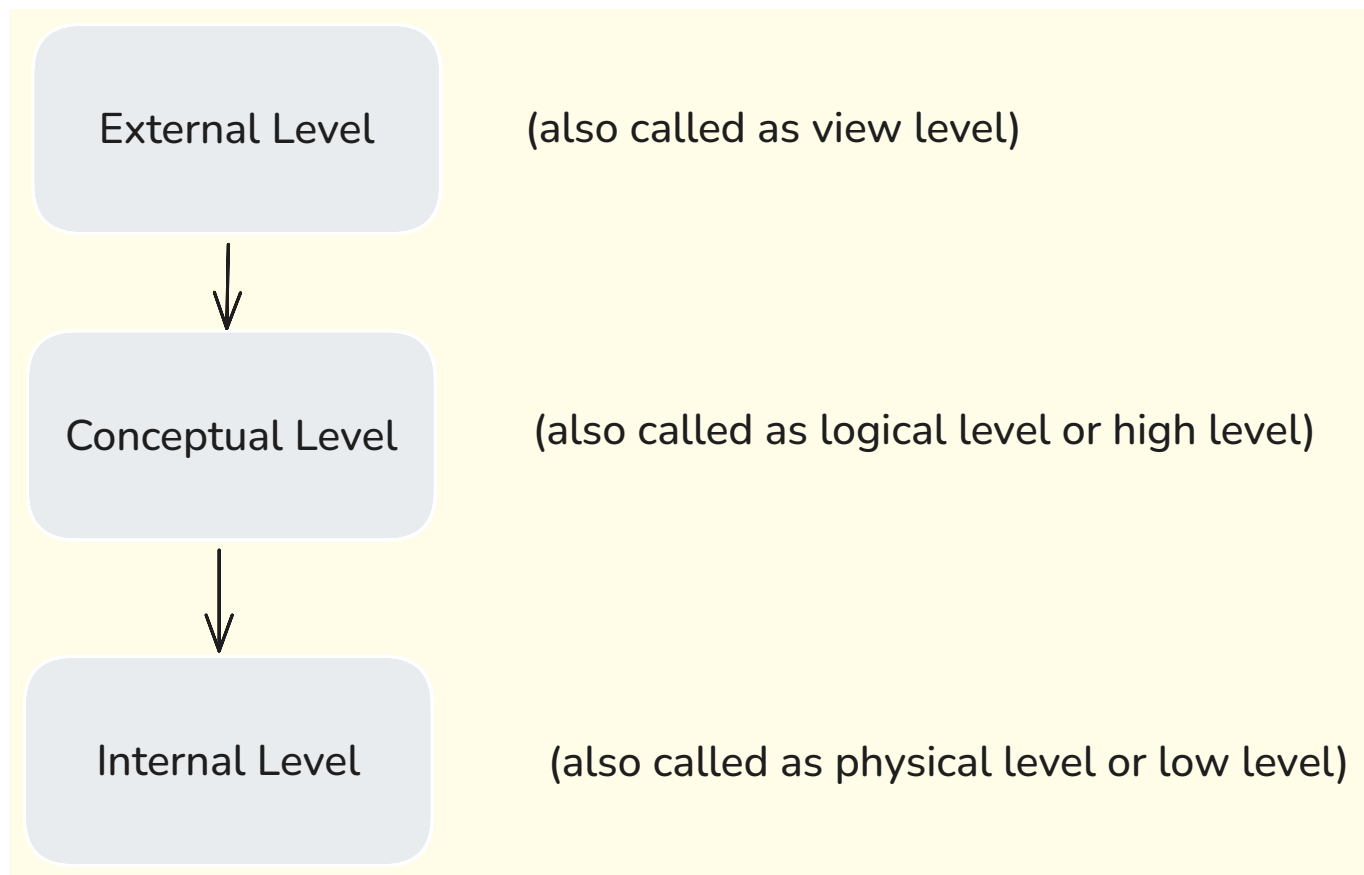
Characteristics of Database

- **Organized Structure**
 - Data is organized into a structured manner using tables, rows, and columns.
 - This organization makes it easier to query and manipulate the database.
- **Data Integrity**
 - Database enforces constraints and rules to maintain data accuracy and consistency.
- ****Data Redundancy**
 - Redundancy ensures that we have enough backups of the original data so that in case of a downtime or failure, we can quickly recover the data and get the systems back to normal.
 - It involves creating timely backups, deleting outdated backups, etc.
- **Data Consistency**
 - Database consistency ensures that data is consistent across multiple tables and users.
 - If multiple users are accessing the same database at the same time then it's very crucial to keep the order of operations.
- **Data Security**
 - Enforce security measures to protect database from unauthorized access.
 - This feature includes user authentication, access control, and encryption.
- **Data Sharing**

- Database sharing allows multiple users and applications to access and share data simultaneously.
-

Database Architecture

It is a blueprint for building, outlining how all the different components fit together and interact to create a functional and efficient systems.



- **External Level**
 - This is the level closest to the users.
 - Each user has a different view of the database, each tailored to the needs of a specific group or

applications.

- A view is a subset of the database that is presented to a user in a way that is relevant to their tasks.
 - **Conceptual Level**
 - This level provides a unified, logical view of the entire database.
 - It describes the overall structure of the database, including its entities, attributes, and relationships without specifying how data is physically stored.
 - It is basically a blueprint of the database.
 - **Internal Level**
 - This is the level closest to the physical storage of the data.
 - It describes how the data is physically stored on the storage devices, including file organizations, data structure and access methods.
-

Data Independence

The three level architecture promotes data independence, which means that changes at one level do not necessarily affect the other level.

Types of Data Independence:

1. Logical Independence

- refers to the changes at the conceptual schema.
- e.g.: adding a new attribute does not change how the user performs a query

2. Physical Independence

- refers to the changes in the internal schema.
- e.g.: changes the storage medium do not require changes to the conceptual schema.

Here is a real life analogy to understand this whole concept:

Imagine a library.

Different readers view books based on their own taste. An otaku lover would love reading mangas whereas a sci-fi lover would love looking at sci-fi books while a novel reader would like to read more novels.

Notice how every reader looks at the books in their own way. Well, this is the **external** level.

Now, every book in a library needs to be recorded in the catalogue system so that it can be tracked. When the books first arrived there, the catalogue system stored attributes like title, author, ISBN, publication date, etc. and stored it in a centralized database. Now using this database we can filter out books based on the author, publication date, etc. This is the **logical** or **conceptual** level.

At last, how the books are arranged refer to the **physical** level.

Now, let's expand this analogy to see how data independence comes into play.

Let's say we decided add one more attribute to the book records. This wouldn't affect the external level meaning it won't change the way readers look at books. This is what's called **Logical Independence**.

Similarly if we decide to rearrange the books, it won't change how the books are stored in catalogue system. This is referred to as **Physical Independence**.

Databases Languages

A database language is a specialized programming language used to interact with a database management system (DBMS).

There are 3 types of Database Languages:

1. DDL - Database Definition Language
2. DML - Database Manipulation Language
3. DCL - Data Control Language

DDL

- stands for Database Definition Language
- It is used to define the structure of the database, including creating, altering, and dropping tables.
- It includes commands like:
 - CREATE
 - ALTER
 - DROP
 - TRUNCATE
 - RENAME

DML

- stands for Database Manipulative Language
- It is used to manipulate the data stored within the database.
- It includes commands like:
 - `SELECT`
 - `INSERT`
 - `UPDATE`
 - `DELETE`

DCL

- stands for Database Control Language
 - It is used to define control for the database, basically define who have access to the database and its contents.
 - It includes commands like:
 - `GRANT`
 - `REVOKE`
-

Database Interfaces

Database Interfaces refers to the mechanisms and tools through which users and application programs interact with DBMS.

Primary types of DBMS interfaces are:

1. CLI

- stands for Command Line Interface.

- It is a text based interface where users type commands to interact with the DBMS.

2. **GUI**

- stands for Graphical User Interface.
- A visual interface that allows users to interact with the DBMS using windows, icons, and menus.

3. **PLI**

- stands for Programming Language Interface.
- Libraries and APIs that allow application programs written in various programming languages to interact with DBMS.
- Example: JDBC (Java Database Connectivity), SQLITE3 in Python

4. **Web Based Interfaces**

- Interface that are accessed through a web browser providing a convenient way to manage databases over the internet.
- Example: phpMyAdmin, Oracle Application Process

5. **APIs**

- stands for Application Program Interface
- They are sets of functions and procedures that allow software applications to interact with DBMS.
- Example: RESTful APIs, GraphQL APIs

6. **Natural Language Interface**

- Interfaces that allow users to interact with DBMS using natural language queries.
- They convert user queries into runnable code which then executes the tasks on user's behalf. It

eliminates the need for user to be well versed in database languages.

- Example: IBM Watson, Gemini API

Database Schemas

A database schema is essentially the blueprint or structure of a database. It defines how the data is organized within the database, including the tables, columns, relationships, constraints, and other elements.

Instance

In terms of database, an instance refers to a specific, running copy of a database management system (DBMS).

Here's an example of how user view is different from admin view.

Customer View



Garam Masala

a very hot masala (test product)

Rs. 100 / kg

 Add to Cart

Admin View

#	Name	Price	Limited Time	Show in Shop	Actions	
1	Garam Masala	Rs. 100	No	Yes	 Edit	Delete
2	tt	Rs. 34	Yes	Yes	 Edit	Delete

Database View

	id varchar	name text	price numeric	image text[]	description text	limited_time bool	display_shop bool
	1	Garam Masala	100	["https://arwzmtvflqftucdiawi.supabase.c	a very hot masala (test product)	FALSE	TRUE
	3	tt	34	["https://arwzmtvflqftucdiawi.supabase.c	gg	TRUE	TRUE